



explain chain of responsibility design pattern



Chain of Responsibility Design Pattern

Chain of Responsibility (CoR) is a **behavioral design pattern** where a request is passed through a **chain of handlers**, and each handler gets a chance to process the request.

Instead of one class deciding **who should handle the request**, the request moves through multiple handlers until one of them handles it.

Simple real-world example

Imagine an IT support system:

```
graph TD; A[User raises issue] --> B[L1 Support]; B --> C[L2 Support]; C --> D[L3 Support]; D --> E[Development Team];
```

- L1 can solve simple issues → handles it.
- If L1 cannot solve it → passes to L2.
- L2 cannot solve it → passes to L3.
- L3 cannot solve it → passes to Development.

That's Chain of Responsibility.

1. Basic Structure

There are usually four parts:

```
graph TD; A[Client] --> B[Handler 1 → Handler 2 → Handler 3 → Handler 4];
```

Each handler contains:

1. Its own processing logic.

2. A reference to the **next handler**.

For example:

```
</> Java
abstract class Handler {

    protected Handler next;

    public void setNext(Handler next) {
        this.next = next;
    }

    public abstract void handle(Request request);
}
```

Then:

```
Handler A
  ↓
Handler B
  ↓
Handler C
```

2. Simple Java Example

Suppose we have an expense approval system.

Different managers can approve different amounts:

```
Employee
  ↓
Manager      → up to ₹10,000
  ↓
Director     → up to ₹50,000
  ↓
VP           → up to ₹1,00,000
```

Request

```
</> Java
class ExpenseRequest {

    private double amount;
```

```
public ExpenseRequest(double amount) {  
    this.amount = amount;  
}  
  
public double getAmount() {  
    return amount;  
}  
}
```

Handler

</> Java



```
abstract class ExpenseHandler {  
  
    protected ExpenseHandler nextHandler;  
  
    public void setNext(ExpenseHandler nextHandler) {  
        this.nextHandler = nextHandler;  
    }  
  
    public abstract void approve(ExpenseRequest request);  
}
```

Manager

</> Java



```
class Manager extends ExpenseHandler {  
  
    @Override  
    public void approve(ExpenseRequest request) {  
  
        if (request.getAmount() <= 10000) {  
            System.out.println("Manager approved");  
        } else if (nextHandler != null) {  
            nextHandler.approve(request);  
        }  
    }  
}
```

Director

</> Java



```
class Director extends ExpenseHandler {  
  
    @Override
```

```

public void approve(ExpenseRequest request) {

    if (request.getAmount() <= 50000) {
        System.out.println("Director approved");
    } else if (nextHandler != null) {
        nextHandler.approve(request);
    }
}
}

```

VP

```

</> Java
class VP extends ExpenseHandler {

    @Override
    public void approve(ExpenseRequest request) {

        if (request.getAmount() <= 100000) {
            System.out.println("VP approved");
        } else {
            System.out.println("Cannot approve");
        }
    }
}

```

Creating the chain

```

</> Java

Manager manager = new Manager();
Director director = new Director();
VP vp = new VP();

manager.setNext(director);
director.setNext(vp);

```

Now:

```

</> Java

manager.approve(new ExpenseRequest(40000));

```

Flow:

```

Manager
↓

```

```
₹40,000 > ₹10,000
↓
Director
↓
₹40,000 <= ₹50,000
↓
Approved
```

3. Important Point

The client doesn't need to know which handler will process the request.

The client simply does:

```
</> Java
```

```
manager.approve(request);
```



The chain takes care of the rest.

That's the major benefit.

4. How the flow works

Suppose:

```
</> Java
```

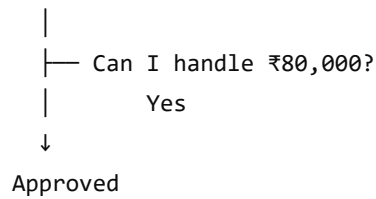
```
manager.approve(new ExpenseRequest(80000));
```



Execution:

```
Client
↓
Manager
|
|— Can I handle ₹80,000?
|   No
↓
Director
|
|— Can I handle ₹80,000?
|   No
↓
VP
```





The request travels from one handler to another.

5. Chain of Responsibility vs normal if-else

Without the pattern:

`</>` Java

```

if (amount <= 10000) {
    managerApprove();
} else if (amount <= 50000) {
    directorApprove();
} else if (amount <= 100000) {
    vpApprove();
}
  
```

This works, but the logic becomes centralized.

With Chain of Responsibility:

Manager → Director → VP

Each class owns its own responsibility.

So if you add:

CEO

you can add another handler:

Manager → Director → VP → CEO

without putting everything into one large if-else .

6. Two common behaviors

There are two common variations.

A. First handler processes it

```

Handler A
  ↓ cannot handle
Handler B
  ↓ can handle
STOP

```



Only one handler processes the request.

B. Multiple handlers process it

```

Handler A → processes
  ↓
Handler B → processes
  ↓
Handler C → processes

```



This is also common when the chain represents a sequence of processing steps.

7. Very important: Chain ≠ always "one handler only"

This is a common interview confusion.

Chain of Responsibility means:

A request has multiple potential handlers arranged in a chain.

It does **not** strictly mean only one handler must execute.

Depending on implementation:

```

Request
  ↓
A → B → C

```



could mean:

First capable handler:

```

A ❌ → B ✅ → STOP

```



or:

Pipeline:

A → B → C



where every handler performs some processing.

8. Real-world Java/Spring example

This pattern is very relevant to **Spring Security**.

A simplified request flow looks like:

```
HTTP Request
  ↓
Security Filter 1
  ↓
Security Filter 2
  ↓
Security Filter 3
  ↓
Security Filter 4
  ↓
Controller
```



For example:

```
Request
  ↓
BearerTokenAuthenticationFilter
  ↓
AuthorizationFilter
  ↓
ExceptionTranslationFilter
  ↓
...
```



Each filter can inspect/process the request and either:

- handle something itself, or
- continue the chain.

Conceptually:

```
</> Java

public void doFilter(request, response, chain) {

    // process something
```




```
chain.doFilter(request, response);  
}
```

This is closely related to the **chain/pipeline form** of Chain of Responsibility.

9. Advantages

1. Reduces coupling

Client doesn't need to know the exact handler.

Client → Handler chain



rather than:

Client → Manager
Client → Director
Client → VP



2. Easy to add/remove handlers

You can change:

A → B → C



to:

A → B → D → C



without changing the client.

3. Each handler has a focused responsibility

AuthenticationHandler
AuthorizationHandler
LoggingHandler
ValidationHandler



rather than one giant class.

4. Good for processing pipelines

Especially useful when requests need to pass through multiple processing stages.

10. Disadvantages

1. Request may not be handled

For example:

A → B → C



If none can process it:

A ✗
↓
B ✗
↓
C ✗
↓
No handler



You need to decide what happens.

2. Debugging can become difficult

A long chain can be:

A → B → C → D → E → F → G



It may not immediately be obvious where the request was modified or rejected.

3. Order matters

Changing:

A → B → C



to:

B → A → C



can change behavior.

This is particularly important in filter chains.

11. When should you use it?

Use Chain of Responsibility when:

- Multiple objects **could handle a request**.
- You don't want the client to decide which object handles it.
- Processing naturally happens in a **sequence**.
- You want to dynamically change the chain.
- You have many conditional handlers.

Typical examples:

Logging
Validation
Authentication
Authorization
Exception handling
Approval workflows
Request processing
Event processing
Support escalation



Interview definition

If an interviewer asks "**What is Chain of Responsibility?**", you can say:

Chain of Responsibility is a behavioral design pattern where a request is passed through a chain of handlers, and each handler can either process the request or pass it to the next handler. It reduces coupling between the sender and the object that ultimately handles the request.

For your **Java/Spring 7–8 year experience preparation**, the important part is to understand **how the chain is built, how next works, how termination happens, and how Spring Security filters use the chain/pipeline concept**.



Yes. For a **7–8 year Java/Spring interview**, you don't need to go extremely deep, but there are a few important areas beyond the basic definition.

Must know

1. Structure of the pattern

- Handler interface/abstract class

- next handler
- Client
- Concrete handlers

2. Two implementation styles

- **Stop after handling**

A → B → C
 ↑
 handles → STOP



- **Continue after handling**

A → B → C



All handlers may process the request.

3. How the chain is constructed

- Manually
- Through dependency injection
- Dynamically based on configuration

4. Termination

Understand what happens when:

- A handler handles the request.
- No handler can handle it.
- A handler intentionally stops the chain.

5. Ordering

Validation → Authentication → Authorization



VS.

Authentication → Validation → Authorization



can produce different behavior.

6. Chain vs Pipeline

Understand the distinction conceptually:

CoR:
 A → B → C



Often means **one of them may handle the request.**

Pipeline:

A → B → C → D



Usually means **each stage performs processing**.

In real frameworks, the concepts can overlap.

Important for Spring

You should understand how the pattern appears in:

- **Spring Security filter chain**
- Servlet Filter chain
- Spring HandlerInterceptor processing
- Exception handling chains/concepts

For Spring Security specifically, understand:

```
HTTP Request
  ↓
DelegatingFilterProxy
  ↓
FilterChainProxy
  ↓
SecurityFilterChain
  ↓
Security Filters
  ↓
Controller
```



And especially what:

</> **Java**

```
chain.doFilter(request, response);
```



means.

Nice to know

7. CoR vs Strategy Pattern

- Strategy → choose **one** algorithm/implementation
- CoR → pass request through **multiple** possible handlers

8. CoR vs Decorator

Both can look like:

A → B → C



But their intent is different:

- Decorator → add/modify behavior.
- CoR → give handlers an opportunity to process a request.

9. Advantages and disadvantages

10. Real-world examples

11. Thread-safety considerations when handlers maintain state.

12. Dynamic chain construction using Spring dependency injection.

What you DON'T need to spend much time on

For your experience level, you don't need to memorize obscure variations or implement a huge framework from scratch.

Recommended depth:

Definition	✓
Structure	✓
Java implementation	✓
Request flow	✓
Stop/continue behavior	✓
Chain construction	✓
Ordering	✓
CoR vs Pipeline	✓
CoR vs Strategy	✓
CoR vs Decorator	✓
Spring Security connection	✓ ✓
Advanced implementation	Optional



If you understand these, **Chain of Responsibility** is sufficiently covered for your **Spring/Java interview preparation**.



[explain each in detail](#)

Absolutely. For your **7–8 year Java/Spring preparation**, let's cover Chain of Responsibility from the basics to the interview-level concepts, including how it connects to Spring Security.

1. Definition

Chain of Responsibility (CoR) is a **behavioral design pattern** in which a request is passed through a sequence of handlers.

Each handler decides:

1. **Can I handle this request?**
2. If yes → handle it.
3. If no → pass it to the next handler.

Conceptually:

```
Client
|
v
Handler A
|
| cannot handle
v
Handler B
|
| cannot handle
v
Handler C
|
| can handle
v
Handle Request
```

The important idea is:

The sender of the request doesn't need to know which object will ultimately handle it.

2. Why do we need Chain of Responsibility?

Consider this code:

```
</> Java

if (amount <= 10000) {
    manager.approve();
} else if (amount <= 50000) {
    director.approve();
} else if (amount <= 100000) {
    vp.approve();
} else {
    ceo.approve();
}
```

The client has to know:

- Manager
- Director
- VP
- CEO
- Their approval limits

As the system grows, this becomes difficult to maintain.

With CoR:

Manager → Director → VP → CEO



The client only knows the first handler:

`</> Java`

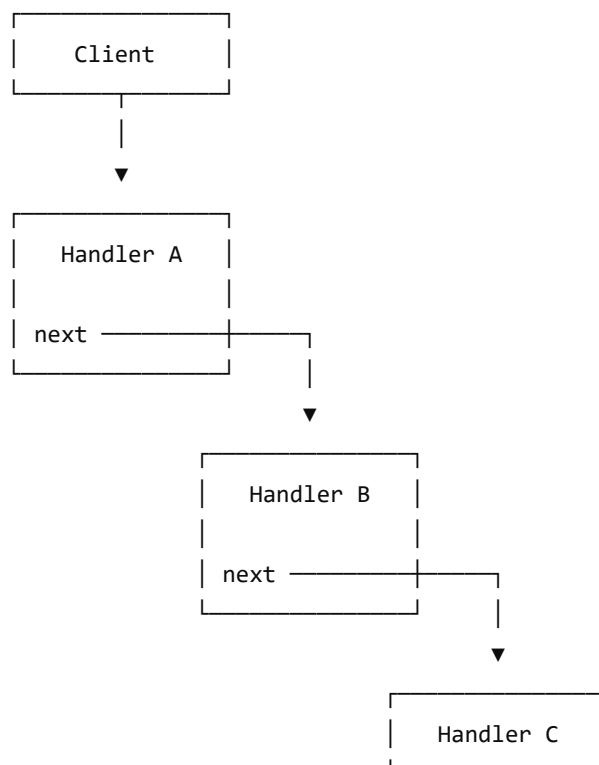
```
manager.approve(request);
```



The chain decides where the request goes.

3. Structure of Chain of Responsibility

There are normally four important components.



Components

1. Client

Creates/sends the request.

2. Handler

Defines the common contract.

Example:

```
</> Java

interface Handler {
    void handle(Request request);
}
```



3. Concrete Handler

Actually implements the processing logic.

```
</> Java

class HandlerA implements Handler {
    // processing logic
}
```



4. Next handler

Each handler maintains a reference to the next handler.

```
</> Java

private Handler next;
```



This creates the chain.

4. Basic Java implementation

Let's build one properly.

Request

```
</> Java
```



```
class Request {  
  
    private int amount;  
  
    public Request(int amount) {  
        this.amount = amount;  
    }  
  
    public int getAmount() {  
        return amount;  
    }  
}
```

Handler

```
</> Java   
  
abstract class Handler {  
  
    protected Handler next;  
  
    public void setNext(Handler next) {  
        this.next = next;  
    }  
  
    public abstract void handle(Request request);  
}
```

Notice this:

```
</> Java   
  
protected Handler next;
```

This is the key to creating the chain.

5. Concrete Handler

Manager:

```
</> Java   
  
class Manager extends Handler {  
  
    @Override  
    public void handle(Request request) {
```

```
        if (request.getAmount() <= 10000) {  
  
            System.out.println("Manager approved");  
  
        } else if (next != null) {  
  
            next.handle(request);  
        }  
    }  
}
```

Director:

```
</> Java   
  
class Director extends Handler {  
  
    @Override  
    public void handle(Request request) {  
  
        if (request.getAmount() <= 50000) {  
  
            System.out.println("Director approved");  
  
        } else if (next != null) {  
  
            next.handle(request);  
        }  
    }  
}
```

VP:

```
</> Java   
  
class VP extends Handler {  
  
    @Override  
    public void handle(Request request) {  
  
        if (request.getAmount() <= 100000) {  
  
            System.out.println("VP approved");  
  
        } else {  
  
            System.out.println("Cannot approve");  
        }  
    }  
}
```

6. Building the chain

</> Java



```
Manager manager = new Manager();  
Director director = new Director();  
VP vp = new VP();  
  
manager.setNext(director);  
director.setNext(vp);
```

Now:

Manager
↓
Director
↓
VP



The client can simply do:

</> Java



```
manager.handle(new Request(40000));
```

7. How exactly does the request travel?

Suppose:

</> Java



```
new Request(40000)
```

is sent.

Step 1

Client calls:

</> Java



```
manager.handle(request);
```

Manager checks:

```
</> Java
```



```
40000 <= 10000
```

False.

So:

```
</> Java
```



```
next.handle(request);
```

is executed.

Step 2

Director receives it.

```
</> Java
```



```
40000 <= 50000
```

True.

So:

```
Director approved
```



The chain stops.

Complete flow:

```
Client
```

```
↓
```

```
Manager
```

```
↓
```

```
40000 > 10000
```

```
↓
```

```
Director
```

```
↓
```

```
40000 <= 50000
```

```
↓
```

```
Approved
```



8. Stop-after-handling behavior

This is one of the most important concepts.

Suppose:

A → B → C



Request reaches A.

A: Can I handle it? NO



Passes to B.

B: Can I handle it? YES



B handles it.

Then:

STOP



C never receives it.

A → B → C

↑

handles

X



This is common when the requirement is:

Find the first handler capable of handling the request.

Examples:

- Approval hierarchy
- Exception handling
- Authentication mechanism selection
- Support escalation

9. Continue-after-handling behavior

Another implementation is:

</> Java



```
public void handle(Request request) {  
  
    // process request  
  
    if (next != null) {  
        next.handle(request);  
    }  
}
```

Notice something important.

Even after handling the request, it calls:

</> Java



```
next.handle(request);
```

So:

Request



A processes



B processes



C processes



D processes



This is more like a **processing pipeline**.

Examples:

Logging



Validation



Authentication



Authorization



Controller



Every stage may perform some work.

10. Chain of Responsibility vs Pipeline

These concepts are very similar, which is why interviews sometimes confuse people.

Traditional CoR

The question is:

"Who should handle this request?"

A → B → C → D



Potentially only one handler handles it.

Example:

Manager → Director → VP



Pipeline

The question is:

"What processing stages should this request go through?"

A → B → C → D



Multiple/all stages process it.

Example:

Logging
↓
Validation
↓
Authentication
↓
Authorization



Important

Real-world frameworks don't always fit perfectly into one textbook definition.

Spring Security's filter chain is best understood as a **chain/pipeline of filters**, even though it uses the same fundamental idea of passing a request along.

11. How is the chain constructed?

There are several ways.

Manual construction

Like our example:

</> Java

```
manager.setNext(director);  
director.setNext(vp);
```



This is simple but can become cumbersome for large applications.

Constructor-based

You can make the dependency explicit:

</> Java

```
class Manager extends Handler {  
  
    public Manager(Handler next) {  
        this.next = next;  
    }  
}
```



Then:

</> Java

```
VP vp = new VP();  
Director director = new Director(vp);  
Manager manager = new Manager(director);
```



Result:

Manager → Director → VP



12. Dynamic chain construction

Sometimes the chain isn't fixed.

For example:

Configuration:
Logging
Validation
Authentication
Authorization



The application can build the chain according to configuration.

This is useful when different environments require different processing.

For example:

Development:
Logging → Validation

Production:
Logging → Validation → Authentication → Authorization



The client doesn't need to change.

13. Spring Dependency Injection and CoR

Spring makes chain construction much easier.

Suppose we have:

```
</> Java

@Component
class ValidationHandler {
}
```



```
</> Java

@Component
class AuthenticationHandler {
}
```



```
</> Java

@Component
class AuthorizationHandler {
}
```



Spring can manage these objects and their dependencies.

In more advanced implementations, handlers can be injected as a collection:

</> **Java**



```
@Autowired  
private List<Handler> handlers;
```

Then the application can process:

</> **Java**



```
for (Handler handler : handlers) {  
    handler.handle(request);  
}
```

The exact implementation depends on how the application wants to define ordering and termination.

14. Ordering

Ordering is extremely important.

Suppose we have:

Validation
Authentication
Authorization



Normally, you don't want authorization to happen before authentication.

Why?

Because authorization needs to know:

Who is the user?

So:

Request
↓
Authentication
↓
Authorization



makes conceptual sense.

If you change it:

```
Request
↓
Authorization
↓
Authentication
```



the behavior can be completely different.

Therefore, in chain-based systems:

The order of handlers can be part of the application's correctness.

15. What happens when nobody handles the request?

Suppose:

```
A → B → C
```



And:

```
A ✗
B ✗
C ✗
```



You need a termination strategy.

Option 1: Do nothing

```
</> Java

if (next != null) {
    next.handle(request);
}
```



If there is no next handler, simply return.

Option 2: Throw exception

```
</> Java

throw new IllegalStateException("No handler found");
```



Option 3: Default handler

A → B → C → DefaultHandler



The default handler handles anything not processed earlier.

This is often a useful design.

16. What if next is null?

This is another common interview question.

You need to explicitly define the behavior.

For example:

```
</> Java

if (next != null) {
    next.handle(request);
} else {
    System.out.println("No handler available");
}
```



Or:

```
</> Java

if (next == null) {
    throw new RuntimeException("Request cannot be handled");
}
```



The important point isn't the exact implementation.

It's that:

The end of the chain must have defined behavior.

17. Chain of Responsibility vs Strategy

This is an important interview comparison.

Strategy

Strategy means:

Choose one algorithm/implementation.

Example:

Payment

↓

Choose strategy

CreditCardStrategy

UPIStrategy

PayPalStrategy



You choose one:

</> Java

```
PaymentStrategy strategy = new UpiStrategy();
```



Chain of Responsibility

You have multiple potential handlers:

Handler A → Handler B → Handler C



The request moves through them until the appropriate handler processes it.

Key difference

Strategy:

Choose ONE strategy



CoR:

Pass request through handlers



18. Chain of Responsibility vs Decorator

These can look almost identical structurally.

Both may look like:

A → B → C



But the **intent** is different.

Decorator

Decorator adds behavior to an object.

For example:

Coffee
↓
MilkDecorator
↓
SugarDecorator



The objective is:

Add responsibilities/features.

CoR

CoR gives handlers a chance to process a request.

Request
↓
ValidationHandler
↓
AuthenticationHandler
↓
AuthorizationHandler



The objective is:

Pass a request through potential handlers.

Interview shortcut

Decorator → enhance object
CoR → process/pass request



19. Chain of Responsibility vs if-else

You should understand when CoR is actually useful.

This:

`</> Java`

```
if (condition1) {  
    ...  
} else if (condition2) {  
    ...  
} else if (condition3) {  
    ...  
}
```



is not automatically bad.

If you only have:

2-3 simple conditions



CoR might be unnecessary.

CoR becomes useful when:

- There are many handlers.
- Each handler has independent responsibility.
- Handlers may change.
- The chain can be reordered.
- You want to avoid a central class knowing every handler.

Don't use a design pattern just to replace a small `if-else`.

20. Real-world example: Support system

Imagine:

```
Customer  
↓  
L1 Support  
↓  
L2 Support  
↓  
L3 Support  
↓  
Engineering
```



Customer doesn't decide:

"If issue is database-related, call L3."



Instead:

`</> Java`

```
l1.handle(ticket);
```



L1 determines:

Can I solve it?



If not:

`</> Java`

```
next.handle(ticket);
```



This is a natural CoR use case.

21. Real-world example: Approval workflow

Employee



Manager



Director



VP



CEO



Request:

₹5,000



Manager handles it.

₹40,000



Director handles it.

₹80,000



VP handles it.

This is a classic **first-capable-handler** implementation.

22. Real-world example: Request processing

Consider an HTTP request.

You might have:

HTTP Request



Logging



CORS



Authentication



Authorization



Controller



Each stage can:

- inspect the request,
- modify something,
- reject the request,
- or continue processing.

For example, authentication might say:

No valid authentication



Reject request

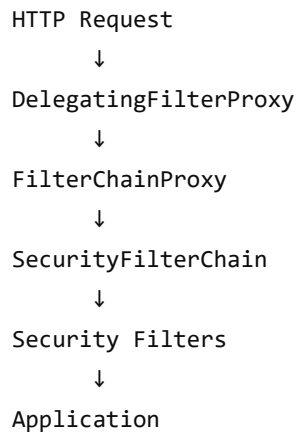


and stop the chain.

23. Spring Security connection

This is especially important for you.

A simplified Spring Security flow:

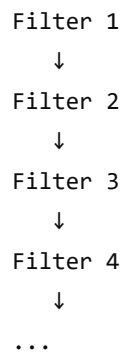


```
graph TD; A[HTTP Request] --> B[DelegatingFilterProxy]; B --> C[FilterChainProxy]; C --> D[SecurityFilterChain]; D --> E[Security Filters]; E --> F[Application];
```

HTTP Request
↓
DelegatingFilterProxy
↓
FilterChainProxy
↓
SecurityFilterChain
↓
Security Filters
↓
Application

Inside the security chain, there are multiple filters.

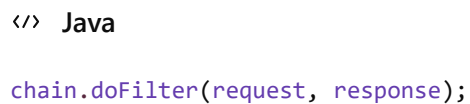
Conceptually:



```
graph TD; A[Filter 1] --> B[Filter 2]; B --> C[Filter 3]; C --> D[Filter 4]; D --> E[...];
```

Filter 1
↓
Filter 2
↓
Filter 3
↓
Filter 4
↓
...

A filter can call:



```
</> Java

chain.doFilter(request, response);
```

That means:

Continue processing with the next filter in the chain.

Or a filter can decide not to continue.

For example:



```
</> Java

if (authenticationFailed) {
    response.setStatus(401);
    return;
}

chain.doFilter(request, response);
```

So:

```

Authentication fails
    ↓
Response 401
    ↓
STOP
  
```



Whereas:

```

</> Java

chain.doFilter(request, response);
  
```



means:

```

Current filter
    ↓
Next filter
  
```



This is a **very important concept** for Spring Security interviews.

24. OncePerRequestFilter connection

You have already studied `OncePerRequestFilter` .

A custom security filter might look like:

```

</> Java

public class JwtFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain) {

        // JWT processing

        filterChain.doFilter(request, response);
    }
}
  
```



The important part here is:

</> Java



```
filterChain.doFilter(request, response);
```

It means:

JWT Filter



Continue



Next Security Filter



If you don't call it:

</> Java



```
return;
```

then subsequent filters don't execute.

This is one of the best practical examples of chain processing in Spring.

25. Thread-safety considerations

This is more advanced but worth knowing at your experience level.

Suppose handlers are Spring singleton beans:

</> Java



```
@Component
class ValidationHandler {

    private int count;
}
```

Spring's default scope is singleton.

That means multiple requests can access the same handler concurrently.

So don't store request-specific mutable state in instance fields:

</> Java



```
private Request currentRequest; // ❌ dangerous
```

Instead, keep request-specific information local:

```
</> Java

public void handle(Request request) {

    // request is local to this invocation
}
```



If shared state is genuinely required, you need appropriate concurrency control.

26. Performance considerations

If you have:

```
A → B → C → D → E → F → G → H
```



and every request travels through all eight handlers, there is processing overhead.

For a small chain this is negligible.

But very large chains can cause:

- extra method calls,
- unnecessary processing,
- harder debugging,
- increased latency.

Therefore:

Keep the chain meaningful and avoid unnecessary handlers.

27. Mutable vs immutable chain

A chain can be configured once:

```
A → B → C
```



and never changed.

This is generally easier to reason about.

Or it can be modified dynamically:

A → B → C



A → D → B → C

Dynamic chains provide flexibility but can introduce:

- thread-safety concerns,
- ordering problems,
- harder debugging.

For most application systems, configuring the chain during initialization is easier to manage.

28. Recursive implementation

The most common CoR implementation looks recursive:

</> Java



```
public void handle(Request request) {  
  
    if (canHandle(request)) {  
        process(request);  
        return;  
    }  
  
    if (next != null) {  
        next.handle(request);  
    }  
}
```

Notice:

</> Java



```
next.handle(request);
```

The current handler calls the next handler.

Conceptually:

```
A.handle()  
  ↓  
B.handle()  
  ↓  
C.handle()
```



This is technically nested method invocation.

For extremely long chains, you should be aware that deep recursive/nested call structures can affect the call stack, though ordinary application chains are usually nowhere near problematic depths.

29. A more reusable Handler implementation

You can also put the common forwarding logic into the base class.

</> Java



```
abstract class Handler {  
  
    private Handler next;  
  
    public Handler setNext(Handler next) {  
        this.next = next;  
        return next;  
    }  
  
    public void handle(Request request) {  
  
        if (canHandle(request)) {  
            process(request);  
        } else if (next != null) {  
            next.handle(request);  
        }  
    }  
  
    protected abstract boolean canHandle(Request request);  
  
    protected abstract void process(Request request);  
}
```

Now concrete handlers only implement:

</> Java



```
canHandle()
```

and:

</> Java



```
process()
```


Example:

</> Java



```
class Manager extends Handler {  
  
    protected boolean canHandle(Request request) {  
        return request.getAmount() <= 10000;  
    }  
  
    protected void process(Request request) {  
        System.out.println("Manager approved");  
    }  
}
```

This reduces duplicate code.

30. Interview questions you should be able to answer

For your experience level, make sure you can answer these:

Basic

1. What is Chain of Responsibility?

Request moves through a chain of handlers until it is handled or the chain ends.

2. Why use it?

To decouple the sender from the receiver/handler and organize processing into independent handlers.

3. What is the role of next ?

It references the next handler in the chain.

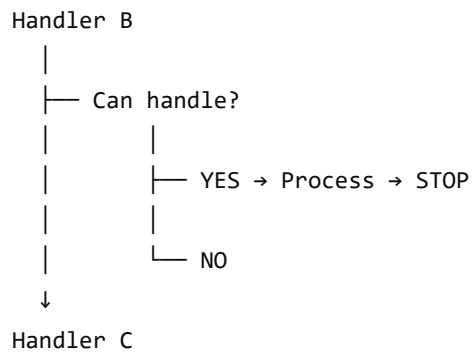
Intermediate

4. What happens if no handler can process the request?

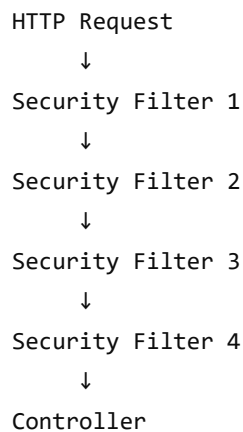
Depends on implementation: ignore, default handler, exception, error response, etc.

5. Can multiple handlers process the same request?

Yes. The implementation can either stop after handling or continue through the chain.



And the Spring Security version:



</> Java

```
filterChain.doFilter(request, response);
```

means:

"I am done with my processing; continue to the next filter."

That connection is probably the **most valuable practical part of Chain of Responsibility for your Spring interview preparation.**



is above details on chanin of responsibility is sufficient

Yes. For your 7–8 year Java/Spring interview preparation, the above Chain of Responsibility coverage is sufficient. ✓

You have covered the important areas:

- Definition and intent

- Problem it solves
- Structure/components
- Handler and next reference
- Concrete handlers
- Building the chain
- Request flow
- Stop-after-handling
- Continue-after-handling
- Chain vs Pipeline
- Chain construction and dynamic chains
- Handler ordering
- End-of-chain behavior
- CoR vs if-else
- CoR vs Strategy
- CoR vs Decorator
- Thread-safety considerations
- Performance considerations
- Recursive implementation
- Real-world examples
- **Spring/Spring Security Filter Chain connection**
- `FilterChain.doFilter()`
- `OncePerRequestFilter` connection
- Interview questions

What you can skip

You don't need to spend more time on:

- Historical background
- Complex variations of CoR
- Rare framework implementations
- Building a custom framework around CoR
- Memorizing UML diagrams
- Extremely advanced theoretical details

Your interview-ready understanding should be

If you can explain this confidently:

Request



Handler A

↓ cannot handle



```
Handler B
  ↓ cannot handle
Handler C
  ↓
handles request
```

and explain:

Each handler either processes the request or passes it to the next handler. The client doesn't need to know which handler ultimately processes it.

And then connect it to:

</> Java



```
filterChain.doFilter(request, response);
```

in Spring Security, you can move on to the next design pattern.

I'd consider Chain of Responsibility DONE for your preparation.



ChatGPT can make mistakes. Check important info.